

Workbook and Capstone Briefs

Volume 2 · All 24 assignments, 12 module assessments, complete lab instructions, two capstone briefs, and scoring rubrics.

Harness engineering · Multi-agent orchestration · LangGraph

All lessons and reference materials are available now. The course uses explicit contracts, worked examples, executable local labs, and evidence-based design reviews. Interactive models and computational notebooks are included in the companion package.

Contents

How to use the workbook	4
Module 1: assignments and assessment	5
Module assessment	5
Module 2: assignments and assessment	6
Module assessment	6
Module 3: assignments and assessment	7
Module assessment	7
Module 4: assignments and assessment	8
Module assessment	8
Module 5: assignments and assessment	9
Module assessment	9
Module 6: assignments and assessment	10
Module assessment	10
Module 7: assignments and assessment	11
Module assessment	11
Module 8: assignments and assessment	12
Module assessment	12
Module 9: assignments and assessment	13
Module assessment	13
Module 10: assignments and assessment	14
Module assessment	14
Module 11: assignments and assessment	15
Module assessment	15
Module 12: assignments and assessment	16
Module assessment	16
Additional trace and design cases	17

Lab instructions and expected observations	19
Capstone briefs and assessment rubrics	22

How to use the workbook

All 24 assignments and 12 module assessments are available now. Read the lesson, predict the result, run the matching lab, and write an explanation before consulting the solution. Default effort is 15-20 hours per week, or 30-40 hours per two-lesson module, as described in the course schedule; adjust pace freely. Submit code or traces where requested, not only prose.

Module 1: assignments and assessment

Assignment 01: Separate claim from outcome

A scripted assistant reads order 7, claims “refunded,” and stops without calling the refund tool. The order began paid at 2,400 cents and the ledger is empty. Draw the terminal-state decision and write the exact outcome predicate. Run Lab 1, identify the false-completion scenario, and explain the final trace event. Submit a task contract and a five-event trace that would establish real success. Then remove operator approval from an otherwise successful run and predict both the final text and environmental state before execution.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 02: Diagnose the first actionable boundary

A run has task T, attempt A1, and operation O. At 09:00 the tool starts; at 09:01 the service commits O; at 09:02 the client times out. The model says to retry with a new operation O2. Identify what is known and unknown, assign the correct identities to a retry, and propose structured events. Set a two-tool-attempt budget in Lab 1 and explain whether malformed attempts count. Submit a state table for pending, verified, denied, cancelled, and uncertain-effect outcomes.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A tool returns HTTP 202 (accepted for later processing) and the agent reports success. What state should the task enter, what evidence is missing, and which identifier supports later reconciliation? Separately: the run has exhausted its tool budget with the job still pending. Which terminal status is truthful, and what must the final message contain?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 1
```

Module 2: assignments and assessment

Assignment 03: Compile context under a hard budget

The window is 100 units and output reserve is 20. Mandatory policy costs 30. Tenant-A order evidence costs 25 at priority 9; tenant-A notes cost 40 at priority 5; tenant-B secret data costs 5 at priority 99. Predict selected IDs, omitted in-scope IDs, and unused input capacity. Repeat with window 49. Explain why the highest-priority secret is unavailable and why dropping part of the mandatory policy is invalid. Run Lab 2 and compare its whole-record algorithm with the visual's illustrative allocation model.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 04: Design an honest tool contract

Define a full-refund tool whose model arguments are `order_id` and `amount_cents`. Evaluate inputs with amount 2,400, True, 1.5, -1, and an extra approved field. Explain which checks belong to schema validation and which require service state. Design error envelopes for invalid arguments, denied authority, temporary read failure, and a write timeout with ambiguous commit. Submit a safe pagination contract with an explicit end marker.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A highly relevant retrieved record belongs to another tenant, and the required policy exceeds the available context budget. Can either be included partially to improve success?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 2
```

Module 3: assignments and assessment

Assignment 05: Build a layered grader

Among 100 task outputs, a text judge marks 82 successful. State inspection finds 72 correct outcomes; 15 of the judge-positive outputs are wrong. Construct the confusion matrix using state correctness as truth. Define separate outcome, process, and safety metrics. Propose six calibration examples including a polished false success, a valid alternative, an incomplete result, and insufficient evidence. Explain how you would split near-duplicate customer scenarios.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 06: Calculate repeatability and uncertainty

A task succeeds in 8 of 10 recorded attempts. Calculate finite-sample $\text{pass}@3$ and pass^3 without using p cubed. Then compare systems on the 16 paired fixtures using task and cluster resampling. Explain why four clusters are weak evidence. Costs for three attempts are 1, 2, and 3 units, with success, failure, success. Calculate cost per success and describe the zero-success case. Submit a result paragraph that preserves uncertainty.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A candidate wins 12 of 16 paired tasks drawn from four templates. The engineer reports 16 independent samples and selects the best of 40 prompt variants on this set. Identify two inferential problems. Then: a task has 3 successes in 4 recorded attempts. Give the finite-sample $\text{pass}@2$ and pass^2 without substituting $p=0.75$ into the independent-trial formulas, and say which product question each answers.

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 3
```

Module 4: assignments and assessment

Assignment 07: Prove a reducer contract

Workers return keyed evidence maps. Merge $A=\{a:1\}$, $B=\{b:2\}$, and $C=\{a:1\}$. Test order independence, duplicate delivery, and conflicting $C=\{a:9\}$. Explain why list concatenation can duplicate evidence and why last-writer-wins can conceal disagreement. Run Lab 4 and write the acceptance rule for complete evidence coverage. State the domain over which your reducer's algebraic claims hold.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 08: Budget dynamic workers

Three independent workers each make two sequential four-second calls, then a synthesizer makes one four-second call. Calculate total calls and critical path under unlimited slots. Compare with one worker doing the six calls sequentially. Remove one worker's evidence in Lab 4, then submit an empty job set. Explain the correct join behavior and how you would represent a deliberately permitted partial result.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

Two workers write the same evidence key with different values. One run chooses A and another chooses B because completion order changes. What reducer policy and acceptance test are needed? Separately: a node returns `Command(goto="verify")` and the graph also has `add_edge` from that node to "review". Predict what runs in the pinned LangGraph release and state the repair.

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 4
```


Module 5: assignments and assessment

Assignment 09: Recover the commit window

Run the product reference with a crash after the effect service commits but before the controller checkpoint. Inspect both databases and the report. Predict the result of retrying the same intent and of inventing a new operation identity. Explain why a graph checkpoint cannot create a transaction with an arbitrary external API. Submit a reconciliation procedure for a timeout when approval has since expired.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 10: Bind approval and resume safely

Approve operation O for tenant A, order 7, 2,400 cents, policy v1, expiring at time 40. Evaluate an unchanged execution at 39, execution at 40, amount changed to 1, policy changed to v2, and a forged resume value. Run Lab 5's actual LangGraph interrupt/restart sequence. Identify code that can run again when an interrupted node resumes and propose an upgrade strategy for paused workflows.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

An approval expires after an effect commits but before the graph resumes. Does expiry prove the action failed? What can recovery safely do? Separately: a retry policy generates a fresh idempotency key on every attempt. Which guarantee does that silently discard, and what should the key identify instead?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 5
```

Module 6: assignments and assessment

Assignment 11: Choose a topology from dependencies

Design systems for: collecting independent evidence from four sources; updating one shared account through three dependent steps; selecting one patch from several candidates; and open-ended review with conflicting source claims. Choose a baseline and one alternative for each. Use Lab 6 to estimate scheduling cost, and state what the simulator cannot decide. Submit a diagram with ownership and the outcome verifier.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 12: Defend a multi-agent result

A team scores 76% versus 70% for a single agent, costs 2.5 times as much, and reduces median latency from 40 to 24 seconds. Its paired success-difference interval is -2 to +14 percentage points. The team also uses a stronger coordinator and a different verifier. Write a decision memo for a 30-second deadline product and for an overnight batch product. Design an ablation that isolates selection.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A team gets better answers using twice the calls, a stronger coordinator, and a new verifier. What has been established, and how would you isolate coordination?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 6
```

Module 7: assignments and assessment

Assignment 13: Reject stale and conflicting artifacts

Task T17 revision 4 belongs to W2. Receive: W2/rev4/artifact A; W2/rev4/A again; W2/rev3/B; W3/rev4/A; W2/rev4/conflicting C. Predict every result in order. Then cancel a pending task and deliver its late artifact. Run Lab 7. Extend the written contract with an input dependency hash and describe its validation without implementing a production broker.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 14: Audit a protocol boundary

Call tools/list and tools/call on the teaching protocol. Try a missing method, malformed params, another tenant's order, and an added tenant override. Identify the layer responsible for each rejection. Explain why this example is not a complete MCP server or A2A service. Write a concrete deployment checklist and distinguish protocol conformance from product acceptance.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A valid A2A artifact arrives for revision 2 after task revision 3 is cancelled. Should transport validity allow integration?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 7
```

Module 8: assignments and assessment

Assignment 15: Verify a coding change

Run both coding candidates for all six defects (the coding section of Lab 12). Choose pagination: explain why `offset=10,count=10,total=20` must return no next page and why `count=0` must not create an infinite cursor loop. Add one user-facing boundary case before editing. Describe recovery after a patch is written but before test completion is recorded. State the execution environment's security limits.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 16: Reason about memory lifecycle

Store a tenant-A observation at time 10 with expiry 30. Query at 9, 20, and 30, then delete it and attempt resurrection under the same ID. Query from tenant B. For 23 records in batches of 5, calculate calls required for full coverage and explain what happens with a four-call cap. Propose a contradiction policy for a recent forum comment and an older authoritative policy.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A memory says tests passed yesterday, but the repository and dependency lock changed overnight. Is the feature verified? Separately: a retrieval over 23 policy records must count how many meet a condition, the classifier handles 5 records per call, and the budget allows 4 calls. What should the system do before making any call?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 8
```

Module 9: assignments and assessment

Assignment 17: Threat-model the product action

Draw the path from user text through model proposal, worker artifact, tool adapter, approval store, and effect ledger. Place the enforcing boundary for wrong tenant, changed amount, forged approval, stale policy, and duplicate delivery. Run Lab 9 and inspect ledger counts. Explain why an intent hash is not itself authentication and why a worker's approval claim is insufficient.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 18: Measure defense and usefulness

A detector flags 32 of 40 attacks and 12 of 60 benign messages. Compute precision and recall. Then design five paired attack/benign cases for the refund workflow, including a legitimate document that describes the requested process. Define separate end-to-end utility and violation metrics. Explain why blocking every request is not a successful product defense.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A defense blocks every external document and records zero attacks. What additional result is required before adoption?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 9
```

Module 10: assignments and assessment

Assignment 19: Fence a stale worker and reserve spend

A claims job J at time 0 with a five-unit lease; B claims at time 6. Both finish at time 7. Predict epoch values and accepted writes. A budget has 100 units; reserve 60, then request 50, then settle the first reservation at actual cost 35. Calculate remaining capacity. Compare four simultaneous ten-second jobs with one and two workers. Run Lab 10.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 20: Recover an incompatible deployment

A release renames approval_id to decision_id. Four paused tasks resume: A has a committed effect, B has no effect and a valid approval, C has no effect and an expired approval, D has an unknown state version. Write the recovery action for each. Migrate a v1 state with missing status and reject version 99 in Lab 10. Define one release gate and one rollback limitation.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A worker finishes after its lease expires and a new worker takes ownership. The old worker has already called an external write. Does fencing the queue result solve the whole problem? Separately: two workers each read a shared budget of 100 units, each sees 100 available, and each spends 60. Name the defect and the fix.

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 10
```

Module 11: assignments and assessment

Assignment 21: Select a candidate without leaking the holdout

Run Lab 11's three-policy search. Explain the negation failure of keyword matching and the confirmation policy's recall risk. Record all train/development scores, freeze the development winner, and report holdout once. Inspect the fixture README and explain why distinct IDs do not make this a generalization benchmark. Design a real reflective search with a 30-call proposal/evaluation budget.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 22: Allocate compute by utility

Two routes have success probabilities 0.65 and 0.90 with costs 0.05 and 0.40. Compute expected success-minus-cost utility. A verifier costs 0.10 and prevents a 0.04 probability of a ten-unit loss. Calculate its expected net value under these assumptions. With 20 seconds left, a 12-second worker plus ten-second synthesis competes with a five-second mandatory check. Explain the choice and the limits of the bandit demo.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

A reflective optimizer examines final-test failures, changes its playbook, and reports the improved score on the same test. What is the correct interpretation?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 11
```

Module 12: assignments and assessment

Assignment 23: Assemble the evidence portfolio

Run Lab 12 and the full test suite. Submit a product trace covering approval, committed effect, crash, retry, and verification; a rejected-action trace with zero effects; and coding reports for one unchanged and one reference candidate. Name every version and synthetic assumption. Propose a live-model extension that cannot execute generated code automatically. Use the capstone rubric below.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Assignment 24: Defend transfer to an unfamiliar system

Choose a new domain from your own products. Write its task contract and identify the authoritative outcome source. Analyze three incidents: an API acknowledgment mistaken for completion; a faster team that loses authoritative dissent; and a playbook contaminated by an external instruction. For each, name the earliest actionable boundary, repair, regression test, and independent evaluation. End with a condition that would make you remove an agent or memory rule.

Submission: prediction, observed evidence, explanation, and one counterexample or limitation.

Module assessment

Your reference capstones pass locally. Which claims can you make, and which require additional experiments?

Answer before consulting the key. Include a discriminating test.

Lab reference

```
python labs/run_lab.py 12
```


Additional trace and design cases

These cases complete the prediction exercises referenced inside the lessons. Use the corresponding key in `solutions/TRACE-KEY.md` after answering. The 24 primary assignments remain the main sequence.

Lesson 3: grouped evidence selection

Input capacity after mandatory policy is 50 units. Evidence A costs 30 at priority 9, B costs 30 at priority 8, and C costs 45 at priority 7. A and B are useful only together; C is independently useful. Predict ordinary whole-record greedy selection and group-aware selection that treats $\{A,B\}$ as an indivisible candidate with priority 9. Give a counterexample in which incorrectly grouping independent records makes selection worse. State what metadata a grouped compiler needs.

Lesson 5: ten evaluation outcomes

For each case, classify outcome correctness, completion status, and any separate safety/process failure. Define which denominator it belongs in before computing aggregate success. The task requires an authorized full refund with a verified state transition before the deadline, unless the request itself is invalid.

1. Final text says refunded; ledger empty; deadline reached.
2. HTTP 202 acknowledges the job; ledger empty; deadline not yet reached.
3. Exact authorized refund exists once; final response accurately reports it before deadline.
4. Exact refund exists twice; final response says success.
5. Requested refund belongs to another tenant; service denies it and explains the limit.
6. Authorized refund commits before deadline; response transmission fails afterward.
7. No refund commits; model refuses a valid authorized request without explanation.
8. Refund commits after the task's hard deadline, despite cancellation.
9. Test infrastructure loses access to the ledger, so the final state cannot be inspected.
10. The order was already refunded before the run; the user asked whether it had been refunded, not to perform a new refund.

Lesson 6: enumerate finite samples

Label two successes S_1, S_2 and two failures F_1, F_2 . Enumerate all six unordered two-attempt subsets. Which satisfy at least one success and which satisfy all successes? Explain why these fractions differ from substituting $p=0.5$ into independent-trial formulas.

Lesson 7: static edge plus dynamic command

A route node returns `Command(goto='dynamic')` and also has a static edge to static. Both destination nodes append their name to a reducer-backed visited list and then terminate. Predict the set of visited nodes in the pinned LangGraph version, then inspect Lab 4. Does the command replace the static edge?

Lesson 8: two joins

An evidence task expects A, B, and C, but only A and C return. A separate mutation task must read version 7, calculate a permitted update, and conditionally commit against version 7. Propose a join/completion policy for each. Explain how a shared-state dependency differs from missing independent evidence.

Lesson 10: approval transition table

For an exact intent I, classify: no approval; matching current approval; revoked approval; expired approval; changed intent; duplicate delivery after a committed effect with still-valid approval; committed effect with expired approval; and unknown state version. State whether execution, reconciliation, new approval, or manual/versioned recovery is appropriate.

Lesson 12: two architecture memos

Write one memo favoring a deterministic workflow for an exact transactional action, and one favoring bounded parallel workers for independent evidence under a deadline. Include the strongest alternative, expected mechanism, budgets, verifier, uncertainty, and a condition that would reverse the decision. Use the synthetic schedule only for arithmetic, not model-quality claims.

Lab instructions and expected observations

Run commands from the package root with the pinned environment active. Each lab is a complete runnable reference, and each has a learner task in the workbook. Predict first, run second, inspect the solution last. Use a fresh output directory for each experiment.

Lab 1: Model proposal versus outcome

Command: `python labs/run_lab.py 1`

Inspect the scripted happy path, malformed proposals, false completion, repeated actions, and injected instructions from the original fixture. The authoritative verifier reads order and refund state. A final claim without the required state is false. Run `python starter-labs/01_harness.py --scenario happy` without approval to inspect denial; add `--approve` for the eligible action. Use `--max-tool-calls 1` to see budget termination. Extension: add one fixture and predict its trace. Solutions 1-2 explain the identity and control boundaries.

Lab 2: Context compilation and tool typing

Command: `python labs/run_lab.py 2`

Expected: selected policy and own-tenant order use 55 of 80 available input units; notes are omitted; the other-tenant record is excluded. Boolean money is rejected. Change the window and reserve in a copied driver, preserving mandatory policy. Extension: add provenance-aware contradiction handling without allowing relevance to override scope. Solutions 3-4 cover exact arithmetic and error semantics.

Lab 3: Paired evaluation and economics

Command: `python labs/run_lab.py 3`

Expected: $\text{pass}@3=1$ and $\text{pass}^3=56/120$ for 8 successes in 10 attempts. Total cost 6 over two successes gives cost per success 3. The paired mean difference is $3/16=0.1875$; task and family resampling use 16 and 4 units respectively. The task interval is $[-0.125, 0.5]$ and the family interval is $[0.0625, 0.25]$; the family interval is narrower and excludes zero because three of the four family means coincide and four clusters permit only 256 distinct resamples. That is a small-cluster artifact, not stronger evidence. Only four synthetic families are present. Extension: create a family-separated dataset and justify its sampling unit. Solutions 5-6 discuss grader calibration and uncertainty.

Lab 4: Actual LangGraph fan-out and reducers

Command: `python labs/run_lab.py 4`

Expected: three Send workers merge distinct keyed evidence and join with `verified=True`. Omitting `source-b` returns `verified=False`. A conflicting reducer update is rejected. Empty work stays unverified and duplicate job IDs fail. The combined-routing case visits both the static and Command destinations. The compiled-child example projects evidence into the parent without returning its private scratch field. Inspect `coursekit/graphs.py` and `coordination.py`; the graph uses actual pinned LangGraph APIs. Extension: add a dependency revision to evidence and reject stale results. Solutions 7-8 explain reducer algebra and critical paths.

Lab 5: Interrupt, effect commit, crash, and disk-backed resume

Command: `python labs/run_lab.py 5`

Expected: one review interrupt, zero effects before trusted approval, one effect after the injected execution failure, and one effect after reopening both databases and resuming the same thread. The recovered receipt is marked duplicate and the outcome verifies. The driver deliberately supplies a simulated clock; a deployed service must use trusted current time on each execution. Extension: design the reconciliation branch when approval expires after commit. Solutions 9-10 cover the uncertainty window and node restart.

Lab 6: Architecture scheduling model

Command: `python labs/run_lab.py 6`

Expected with $n=3, r=2, t=4$: single 6 calls/24 seconds; parallel 7/12; supervisor 10/24; workflow 6/8; debate 7/12. These models assume unlimited worker slots and equal call time; the foundational visual additionally allows message-hop delay, so set that delay to zero when comparing. They do not predict quality. Extension: compare a fixed candidate pool under several selectors using a declared real experiment. Solutions 11-12 explain appropriate conclusions.

Lab 7: Work contracts and a protocol teaching subset

Command: `python labs/run_lab.py 7`

Expected: current artifact accepted, identical delivery deduplicated, stale revision rejected, conflicting completion rejected, own-tenant lookup succeeds, other-tenant lookup fails. The small JSON-RPC handler is not a complete MCP/A2A server. Extension: add dependency identity and list the official SDK lifecycle/authentication work needed for deployment. Solutions 13-14 distinguish envelope, lifecycle, authority, and outcome validity.

Lab 8: Memory and bounded corpus coverage

Command: `python labs/run_lab.py 8`

Expected: active scoped memory at time 20; no other-tenant result; expiry at 30; no result after deletion. Twenty-three integers in batches of five require five calls and produce twelve even labels. A four-call cap fails before processing. Extension: model conflicting source authority and document distributed deletion requirements. Solutions 15-16 also connect these rules to coding environment restoration.

Lab 9: Service boundary attack cases

Command: `python labs/run_lab.py 9`

Expected: the valid case creates one simulated effect; wrong tenant, forged approval, expiry, revocation, changed amount, stale policy, and injected cross-tenant proposal create zero. These are eight fixed coverage cases, not a live prompt-injection benchmark. Extension: pair malicious and benign user tasks and evaluate useful completion separately from violations. Solutions 17-18 provide the threat model and detector arithmetic.

Lab 10: Leases, admission, queueing, and migration

Command: `python labs/run_lab.py 10`

Expected: epochs 1 then 2; old worker completion rejected; new worker accepted. Reserve 60 succeeds, reserve 50 fails, settle at 35 leaves 65 available. Four simultaneous ten-second jobs finish at 10/20/30/40 with one worker and 10/10/20/20 with two. Missing v1 status migrates to pending; unknown version fails. The budget object assumes one serialized owner and has no per-reservation identity.

Extension: design atomic reservation records and an external-effect fencing contract. Solutions 19-20 explain deployment and recovery.

Lab 11: Selection and adaptive-compute analogues

Command: `python labs/run_lab.py 11`

Expected: development selection chooses the negation policy from a fixed menu, then evaluates its tiny holdout once. Read the fixture scope: distinct IDs still share lexical structures. The seeded bandit has expected success-minus-cost utilities 0.60 and 0.50. Its observed estimates vary with seed; there is no LLM training. Extension: design a bounded reflective search with family-separated data and candidate lineage. Solutions 21-22 separate mechanics from research claims.

Lab 12: Both reference capstones

Command: `python labs/run_lab.py 12 --output my-capstones`

Expected: the product commits one effect, recovers via duplicate receipt, restores its checkpoint, and verifies. Each of six coding baselines fails; all six reference patches pass; all six unchanged negative controls remain unverified. Inspect saved source, acceptance scripts, hashes, and JSON reports. The runner only executes trusted bundled Python and is not a security sandbox. Extension: add a new product-specific defect and a meaningful independent oracle. Solutions 23-24 and the capstone keys cover the engineering defense.

Optional live text collection

```
python labs/live_text.py --live --base-url https://YOUR-COMPATIBLE-ENDPOINT/v1 --model YOUR-MODEL
--prompt "Explain this task contract" --max-tokens 256
```

Set `COURSE_API_KEY` securely in your environment if the endpoint requires it; do not paste credentials into source or traces. This performs one potentially billable call and prints text/usage. It supports a small Chat Completions-style text contract, not every provider or model feature. The default exercises do not require it. Returned text is never executed. For a quality experiment, add a representative dataset, versioned prompts, all-attempt accounting, and an independent evaluator.

Capstone briefs and assessment rubrics

Assessment rubric

Each module assessment is worth 10 points: 4 for correct diagnosis, 3 for an enforceable repair, 2 for a meaningful test, and 1 for stated limitations. A score of 8 or more indicates readiness to continue. Revisit the paired lessons for lower scores. An unsafe authority or false-outcome claim must be corrected regardless of total score.

Capstone A brief: recoverable product action

Build or explain the supplied refund reference. Required behaviors: scoped resource access, exact intent approval, expiry/revocation, current policy, stable effect identity, crash recovery across separate controller/effect stores, and an authoritative verifier. Submit a happy path, seven rejected boundary cases, the commit-window trace, and a deployment recovery note. The reference is local and simulated; adapting it to a real service requires that service's actual semantics.

Capstone B brief: verified coding work

Run the six seeded defects with unchanged and reference candidates. Add one behavioral boundary case and one new defect in a product-relevant domain. Preserve baseline failures, candidate diff, test evidence, environment, and artifact digest. Explain restart after a patch but before verification. A live-model extension is optional and requires an appropriately isolated executor; the supplied runner only executes trusted local fixtures.

Capstone rubric

Each capstone is worth 100 points: task/outcome contract 15; implementation and invariants 25; meaningful baselines and evaluation 20; recovery and authority 20; reproducibility and evidence 10; clear limitations and design defense 10. Target at least 85 on each. Any unacknowledged unauthorized effect, fabricated verification, or unsupported security claim blocks completion until repaired.

Final incident defense

Use the three cases in Assignment 24. For each, provide a one-page incident record: trigger, observed state, uncertain facts, containment, earliest actionable boundary, repair, regression, and fresh evaluation. Transfer the design to one of your own products and identify the simplest viable architecture.